KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Warehouse Inventory Monitoring and Stock Reorder System

Submitted by

MITHILA P

25BCS128

Under the Guidance of

Mr.Aravind

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled **"Warehouse Inventory Monitoring and Stock Reorder System"** submitted in partial fulfillment of the requirements for the course **java programming** is a **record of my original work** carried out under the guidance of **Mr.Aravind,** Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

Mithila P

CERTIFICATE

This is to certify that the project entitled **"Warehouse Inventory Monitoring and Stock Reorder System"** is a **bona fide work** carried out by **MITHILA P**, **I-B 2025 B.SC.Computer Science**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **Java programming**

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: coimbatore

Date:

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical Trainer, KGCAS)

Mr.Aravind

Head of the Department
(Technical Head, KGCAS)

Mr.Boopalan

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	1
1.1	Objective	1
1.2	Overview	2
1.3	Features	2
2	**Program Analysis**	3
2.1	Problem Definition	3
2.2	Project Overview	3
2.3	Scopes and Limitations	4
3	**System Design**	5
3.1	Modules	5
4	**Methodology**	**7**
4.1	Problem Analysis	7
4.2	System Design	8
4.3	Program Development	8
4.4	Testing and Validation	9
4.5	Result and Documentation	9
4.6	Algorithm	10
4.7	Flowchart	10
5	**Source Code**	12
6	**Input and Output Screens**	16
7	**Discussion and Conclusion**	17
7.1	Discussion	17
7.2	Conclusion	17
8	**References**	18

ABSTRACT

Abstraction in the Warehouse Inventory Monitoring and Stock Reorder System is used to hide the internal implementation details and show only the essential features of the system. It is implemented using abstract classes and methods, where common properties like item name, ID, quantity, and price are defined in a base class, while specific details are handled in child classes. This helps in organizing the system in a better way by separating what an object does from how it does it. Abstraction also improves security by restricting direct access to unnecessary data and reduces complexity in the code. Overall, it makes the system more flexible, reusable, and easier to maintain and upgrade in future development.

1.INTRODUCTION

The Warehouse Inventory Monitoring and Stock Reorder System is designed to help businesses efficiently manage their inventory and maintain optimal stock levels. Inventory management plays a crucial role in the smooth functioning of warehouses, retail stores, and supply chain operations. Without a proper system, businesses may face issues such as overstocking, understocking, and difficulty in tracking items, which can lead to financial losses and operational inefficiencies.

This system provides a simple and effective solution by automating the process of monitoring inventory levels and generating alerts when stock reaches a minimum threshold. It enables users to store product details, update quantities, and keep track of available stock in real time. By implementing an automated approach, the system reduces the dependency on manual record-keeping and minimizes human errors.

Developed using Core Java, the project applies fundamental programming concepts such as object-oriented design, control structures, and exception handling. The system can be implemented as a console-based application or extended with a graphical user interface for better user interaction.

Overall, this project aims to improve accuracy, save time, and enhance decision-making in inventory management. It serves as a basic yet effective tool for managing warehouse stock and can be further expanded to meet the needs of larger and more complex business environments.

1.1 Objective

The main objective of the Warehouse Inventory Monitoring and Stock Reorder System is to develop a simple and efficient application using Core Java that automates the process of inventory tracking and stock management in a warehouse.

The specific objectives of this project are:

• To design and develop a Java-based inventory monitoring system.
• To maintain accurate records of items, including name, quantity, and stock levels.
• To automate the monitoring of inventory and identify low stock levels.
• To generate alerts when stock reaches the predefined reorder level.
• To reduce manual errors in inventory tracking and management.
• To provide a user-friendly system for easy updating and viewing of stock details.
• To demonstrate the implementation of Core Java programming concepts such as classes, objects, methods, loops, and conditional statements.
• To ensure efficient stock management and timely reordering of items.

1.2 Overview

The Warehouse Inventory Monitoring and Stock Reorder System is a menu-driven application developed using the Java programming language. The system provides a digital platform for managing and tracking inventory in a warehouse efficiently.

The system allows users to add, update, and view product details such as item name, quantity, and stock levels. It continuously monitors the inventory and compares the available stock with a predefined reorder level. When the stock of any item falls below this level, the system generates an alert indicating that the item needs to be restocked.

This system simplifies inventory management by reducing the need for manual record-keeping and minimizing errors in stock tracking. It ensures that businesses maintain adequate stock levels and avoid issues such as stock shortages or overstocking. The application can be implemented as a console-based program or enhanced with a graphical user interface using Java Swing.

This project demonstrates the practical use of Java programming in solving real-world problems related to inventory management and provides an effective solution for small and medium-scale business environments.

1.3 Features

The Warehouse Inventory Monitoring and Stock Reorder System includes the following main features:

• **Item Management –** Allows users to add, update, and delete product details such as item name, quantity, and stock level.
• **Inventory Display** – Displays a list of all available items along with their current stock details.
• **Stock Monitoring** – Continuously monitors inventory levels to track available stock.
• **Reorder Level Setting** – Enables setting a minimum stock level (reorder point) for each item.
• **Automatic Reorder Alert** – Generates alerts when the stock falls below the predefined reorder level.
• **Accurate Record Maintenance** – Maintains organized and error-free inventory records.
• **User-Friendly Interface** – Provides a simple and easy-to-use menu-driven interface.
• **Efficient Stock Management** – Helps in maintaining optimal stock levels and avoiding shortages or overstocking.

2.PROGRAM ANALYSIS

The Warehouse Inventory Monitoring and Stock Reorder System is designed to manage and track inventory efficiently using Java. It stores item details such as name, quantity, and reorder level, and allows users to perform operations like adding, updating, and viewing stock.

The system compares current stock with the reorder level and generates alerts when stock is low. It uses basic Java concepts like loops, conditions, and exception handling to ensure smooth operation. Overall, it reduces manual errors and improves inventory management efficiency.

2.1 Problem Definition

In many small and medium-scale warehouses, inventory management is often performed manually using paper records or basic tools. This traditional method creates several challenges.

Some common problems include:
• Inaccurate tracking of stock levels
• Time-consuming inventory management process
• Difficulty in monitoring multiple items
• Lack of proper inventory records
• Delays in identifying low stock levels

Manual inventory systems also increase the workload for warehouse staff and can lead to issues such as overstocking or stock shortages, affecting overall business operations.

Therefore, there is a need for an automated system that can efficiently monitor inventory, track stock levels, and generate timely reorder alerts.

The Warehouse Inventory Monitoring and Stock Reorder System is designed to solve these problems by providing a computerized solution that automates inventory tracking and stock management.

2.2 Project Overview

The Warehouse Inventory Monitoring and Stock Reorder System is a Java-based application developed to manage and track inventory efficiently in a warehouse environment. It provides a structured and automated way to handle stock details and ensures proper monitoring of available items.

Main functionalities:

• Add new product details (item name, quantity, reorder level)
• Update existing stock information
• View complete inventory in a structured format
• Monitor stock levels continuously
• Generate alerts when stock reaches below reorder level
• Maintain accurate inventory records using file handling (if implemented)
• Load and save inventory data for future use

The system helps in reducing manual work by automating inventory tracking and improves accuracy in stock management. It ensures timely reordering of items and prevents issues such as stock shortages or overstocking.

2.3 Scopes and Limitations

Scope:

The Warehouse Inventory Monitoring and Stock Reorder System is useful for managing and tracking stock in small and medium-scale warehouses. It helps add, update, and view product details, and generates alerts when stock reaches the reorder level. The system can also be extended with database and GUI support for better functionality.

Limitations:

• Not suitable for very large-scale warehouse systems
• Basic user interface in console-based version
• Requires manual data entry
• Lacks advanced features like automation, analytics, and real-time tracking
• No database integration in basic version

3.SYSTEM DESIGN

System design is the stage where the structure and components of the system are planned before the actual implementation. In this phase, the system is divided into different modules, and the workflow of the application is defined. Proper system design helps in organizing the program efficiently and makes development easier.

The Warehouse Inventory Monitoring and Stock Reorder System is designed as a menu-driven application where users can add products, update stock details, view inventory, and monitor stock levels automatically.

3.1 Modules

The main modules of the Warehouse Inventory Monitoring and Stock Reorder System are explained below:

1. Product Management Module

This module is responsible for managing product details such as item name, quantity, and reorder level. It helps in maintaining accurate inventory records.

Example Product List:

1. Rice – 50 units – Reorder Level: 10
2. Sugar – 30 units – Reorder Level: 5
3. Oil – 20 units – Reorder Level: 5
4. Wheat Flour – 40 units – Reorder Level: 10
5. Soap – 25 units – Reorder Level: 8

The product list is displayed when the program starts so that the user can easily view and manage available stock items.

2. Stock Management Module

This module allows the user to update product stock by adding or reducing quantities based on inventory changes.

Functions of this module include:
• Accepting user input for stock updates
• Adding new stock to existing items
• Reducing stock when items are used or sold
• Storing updated inventory details

The system allows users to manage multiple products and keep inventory data up to date.

3. Stock Monitoring Module

This module continuously checks the current stock level of each item.

The system calculates:
• Current stock availability
• Comparison with reorder level
• Identification of low-stock items

The monitoring is performed automatically using Java logic.

4. Reorder Alert Module

This module generates alerts when the stock of any item falls below the predefined reorder level.

The alert includes:
• Item name
• Current stock
• Reorder requirement message

Example Alert Output:
------ STOCK ALERT ------
Rice stock is low (Remaining: 8 units). Please reorder soon!

5. Exit Module

This module allows the user to safely exit the application after completing all inventory operations. When selected, the system terminates the program properly.

4. METHODOLOGY

Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting the system.

For the Warehouse Inventory Monitoring and Stock Reorder System, a structured development approach was followed to ensure that the system effectively manages inventory and meets all required functionalities.

The development process includes the following stages:
• Problem Analysis
• System Design
• Program Development
• Testing and Validation
• Result and Documentation

4.1. Problem Analysis

In this phase, the problem is studied and analyzed in detail. The requirements of the system are identified based on the needs of warehouse inventory management operations.

The main problems identified in manual inventory systems are:
• Manual tracking of stock may lead to errors in inventory records.
• Updating stock details manually takes more time.
• Difficulty in monitoring multiple product quantities efficiently.
• Lack of automated alerts for low stock or reorder levels.

Based on these problems, the system requirements were defined to develop an automated inventory monitoring application that simplifies stock management and improves accuracy.

4.2. System Design

In this stage, the overall structure of the system is designed before coding begins.

The system is divided into modules such as:
• Product Management Module
• Stock Management Module
• Stock Monitoring Module
• Reorder Alert Module
• Exit Module

The program flow and structure are designed so that users can easily interact with the system and efficiently manage inventory operations.

4.3. Program Development

After designing the system, the program is implemented using the Java programming language.

The following steps were followed during development:
• Creation of classes and methods
• Implementation of menu-driven logic
• Writing code for inventory management operations
• Handling user input for product details and stock updates
• Implementing stock monitoring and reorder alert logic
• Displaying inventory records and system outputs

Core Java concepts such as classes, objects, loops, conditional statements, arrays/collections, and methods were used during development.

4.4. Testing and Validation

Testing ensures that the program works correctly and produces the expected results.
Different test cases were used to check the functionality of the system.

Example Test Cases:

Test Case	Expected Result
Add new product	Product added successfully
Update stock quantity	Stock updated correctly
View inventory	All items displayed properly
Check reorder level	Low stock alert generated correctly
Invalid input	Error handled properly

This phase ensures that the program runs without errors and produces accurate inventory management results.

4.5. Result and Documentation

After successful testing, the Warehouse Inventory Monitoring and Stock Reorder System produces accurate results by managing and updating inventory efficiently. The system correctly stores product details, updates stock levels, and generates alerts when items reach the reorder level.

The results are shown through program execution with proper input and output displays.
Documentation is prepared to explain the system design, working, and functionality, making it easy to understand the overall project process.

4.6. Algorithm

Warehouse Inventory Monitoring and Stock Reorder System

1. Start the program.
2. Display the main menu options.
3. Accept user choice from the menu.
4. If the user selects “Add Product”, input product name, quantity, and reorder level.
5. Store the product details in the system.
6. If the user selects “Update Stock”, modify the existing quantity.
7. Display all product details when “View Inventory” is selected.
8. Compare each product’s quantity with its reorder level.
9. If quantity is less than reorder level, generate a low-stock alert.
10. Repeat the process until the user selects “Exit”, then stop the program.

4.7. Flowchart

Warehouse Inventory Monitoring and Stock Reorder System

```
(Start)
↓
Display Main Menu
↓
User Selects Option
↓
┌──────────────────────────────────┐
│ Decision: Which option?          │
└──────────────────────────────────┘
↓
```

• Add Product

↓

Enter Product Details (Name, Quantity, Reorder Level)

↓

Store Data

↓

Return to Menu

• Update Stock

↓

Enter Product Name

↓

Update Quantity

↓

Return to Menu

• View Inventory

↓

Display All Product Details

↓

Return to Menu

• Check Stock Level

↓

Compare Quantity with Reorder Level

↓

```
┌──────────────────────────────────┐
│ Quantity < Reorder Level?        │
└──────────────────────────────────┘
```

↓ Yes → Generate Reorder Alert → Return to Menu

↓ No → No Alert → Return to Menu

• Exit

↓

Stop Program

(End)

5.SOURCE CODE

```
import java.util.Scanner;


class Item {
    String name;
    int quantity;
    int reorderLevel;


    Item(String name, int quantity, int reorderLevel) {
        this.name = name;
        this.quantity = quantity;
        this.reorderLevel = reorderLevel;
    }


    void display() {
        System.out.println("Item: " + name + ", Quantity: " + quantity + ", Reorder Level: " +
reorderLevel);
    }


    void checkReorder() {
        if (quantity <= reorderLevel) {
            System.out.println(" Reorder needed for: " + name);
        }
    }
}
```

```
public class WarehouseSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Item[] inventory = new Item[10];
        int count = 0;
        int choice;

        do {
            System.out.println("\n--- Warehouse Menu ---");
            System.out.println("1. Add Item");
            System.out.println("2. View Items");
            System.out.println("3. Check Low Stock");
            System.out.println("4. Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();
            sc.nextLine();
            switch (choice) {
                case 1:
                    if (count < inventory.length) {
                        System.out.print("Enter item name: ");
                        String name = sc.nextLine();

                        System.out.print("Enter quantity: ");
                        int qty = sc.nextInt();
```

```
System.out.print("Enter reorder level: ");
            int reorder = sc.nextInt();

            inventory[count] = new Item(name, qty, reorder);
            count++;

            System.out.println(" Item added successfully!");
        } else {
            System.out.println("Inventory full!");
        }
        break;

    case 2:
        System.out.println("\n--- Inventory List ---");
        for (int i = 0; i < count; i++) {
            inventory[i].display();
        }
        break;

    case 3:
        System.out.println("\n--- Low Stock Items ---");
        for (int i = 0; i < count; i++) {
            inventory[i].checkReorder();
        }
        break;
```

```
case 4:
                System.out.println("Exiting system...");
                break;

            default:
                System.out.println("Invalid choice!");
        }

    } while (choice != 4);

    sc.close();
  }
}
```

6.INPUT AND OUTPUT SCREENS

```
----- Warehouse Inventory System -----

Enter product name: Rice
Enter quantity: 50
Enter reorder level: 20

Stock is sufficient.
Data saved successfully.
---------------------------------

Enter product name: Sugar
Enter quantity: 10
Enter reorder level: 15

⚠️ Reorder Required!
Data saved successfully.
---------------------------------

Current Inventory:

Product: Rice    Quantity: 50
Product: Sugar   Quantity: 10 (Low Stock)
```

7. DISCUSSION & CONCLUSION

7.1 Discussion

The Warehouse Inventory Monitoring and Stock Reorder System was developed to automate and improve the process of managing warehouse stock. The system allows users to add inventory items, update quantities, and monitor stock levels efficiently. It also helps in identifying low-stock items and suggests reorder when the quantity goes below a defined limit.

During the development process, Core Java programming concepts such as loops, conditional statements, methods, arrays/collections, and object-oriented programming principles were applied. The system successfully performs inventory tracking, stock updates, and reorder checks accurately.

The system was tested with different inputs to ensure correct functionality of stock management and reorder alerts. The results show that the system reduces manual workload, minimizes human errors, and improves the efficiency of inventory control in a warehouse environment.

7.2 Conclusion

The Warehouse Inventory Monitoring and Stock Reorder System successfully demonstrates the practical application of Core Java programming concepts in developing a real-world inventory management application.

The system automates warehouse stock monitoring, reduces errors in manual tracking, and ensures timely reordering of items when stock levels are low. It improves efficiency and helps maintain proper inventory control.

This project helps students understand how Java can be used to design and implement practical business systems. The system can be further enhanced by adding features such as database connectivity, graphical user interfaces (GUI), barcode scanning, and real-time inventory updates.

8.REFERENCES

The following resources were used during the development of this project:

Books

• Herbert Schildt – *Java: The Complete Reference*

Websites

• https://www.oracle.com/java
• https://www.geeksforgeeks.org
• https://www.w3schools.com
• https://www.javatpoint.com